UNIVERSIDAD INTERNACIONAL DEL ECUADOR

CARRERA DE INGENIERÍA EN INTELIGENCIA ARTIFICIAL

Evaluación en Contacto con el Docente

ASIGNATURA

LOGICA DE PROGRAMACION

ESTUDIANTE

Alvaro Ivan Celi Paladines

04 DE OCTUBRE DE 2026

Quito – Ecuador

Introducción

Aprender programación exige convertir reglas expresadas en lenguaje cotidiano en instrucciones que una computadora pueda ejecutar. En Piedra, Papel o Tijera, el reto es representar las nueve combinaciones posibles, validar los datos y controlar cuándo comienza y termina una partida. Una respuesta inmediata permite observar la relación entre la entrada y la decisión del programa.

Objetivo general

Desarrollar un juego local en Python y Tkinter que aplique lógica de programación y explique el resultado de cada partida, integrando el análisis, el diseño y la implementación trabajados en la asignatura.

Objetivos específicos

- Representar la funcionalidad mediante diagramas.
- Validar que el jugador ingrese 1, 2 o 3.
- Comparar la jugada del usuario con una elección aleatoria.
- Mostrar ganador o empate y la regla aplicada.
- Permitir nuevas partidas y salida voluntaria.
- Comprobar la lógica y organizar los archivos para GitHub.

Descripción del problema

Piedra, Papel o Tijera es un juego sencillo, pero para llevarlo a un programa hay que explicar cada paso con claridad. Una persona reconoce las reglas del juego; la computadora necesita instrucciones para saber qué opción se ingresó, cómo compararla y qué resultado mostrar.

El problema consiste en crear un juego que reciba la elección del usuario, seleccione una jugada para la computadora y determine quién gana. También debe avisar cuando se escribe una opción incorrecta y permitir que el jugador decida si quiere continuar o salir.

La propuesta está pensada para una persona que juega contra la computadora. Funciona en el equipo donde se abre y no necesita conexión a Internet durante la partida. No incluye cuentas, partidas entre varios usuarios ni un marcador que guarde los resultados.

Reglas del juego

1 representa Piedra, 2 representa Papel y 3 representa Tijera. Piedra vence a Tijera, Papel vence a Piedra y Tijera vence a Papel. Si ambos eligen lo mismo, hay empate.

Decisiones para desarrollar el programa

Opción considerada	Razón de la elección
Escribir números o usar botones	Se mantiene la entrada por números, como en el diagrama. Así también se puede mostrar qué ocurre cuando alguien se equivoca.

Jugar en una ventana o en texto	Se usan ventanas de Tkinter, como en el Autónomo 2, para pedir la jugada y mostrar los mensajes.
Mantener todo junto o separar tareas	El primer programa estaba en un solo archivo. En la versión final se separan algunas tareas para cumplir lo trabajado en la cuarta unidad.

Relación con los contenidos de la asignatura

El proyecto reúne lo trabajado en las cuatro unidades. Primero se define cómo debe funcionar el juego, después se programa y finalmente se revisa y se organiza el código.

Unidad	Aplicación en el juego
Unidad 1	Elegir el programa, describir sus funciones y preparar los diagramas. El diagrama del Autónomo 1 es la base del proyecto.
Unidad 2	Preparar Python y Tkinter y comenzar a escribir el programa. El Autónomo 2 incluye el avance inicial y una versión que ya permite jugar.
Unidad 3	Usar condiciones para comparar las jugadas y una repetición para iniciar nuevas partidas.
Unidad 4	Separar tareas en funciones pequeñas. Una revisa la opción, otra escribe el nombre de la jugada y otra obtiene el resultado.

Cronograma de ocho semanas

El siguiente cronograma organiza el proyecto desde su planificación hasta la entrega final.

Semana	Actividad
1	Definir el problema y elegir el juego.
2	Preparar los diagramas de funcionamiento y organización.
3	Preparar el entorno de trabajo y el repositorio.
4	Escribir el primer avance del programa.
5	Agregar la comparación y el resultado de la partida.
6	Agregar la repetición y revisar las distintas jugadas.
7	Organizar las funciones y revisar el código.
8	Preparar el documento, la presentación y el video.

Explicación del sistema desarrollado

El juego se desarrolló en Python. Tkinter permite mostrar las ventanas donde se pide la opción del jugador y se presenta el resultado. El programa comienza mostrando las instrucciones: 1 es Piedra, 2 es Papel y 3 es Tijera.

Después, el jugador escribe su opción. Si ingresa otro número, letras o deja la respuesta vacía, aparece un mensaje de error. El programa vuelve a mostrar las instrucciones para que pueda intentarlo otra vez.

Cuando la opción es correcta, la computadora selecciona una jugada al azar. El programa compara las dos opciones. Primero revisa si son iguales; en ese caso muestra empate. Si son diferentes, aplica las reglas para decidir quién ganó.

El resultado muestra los nombres de las dos jugadas y explica la regla aplicada. Por ejemplo, si el jugador elige Piedra y la computadora elige Tijera, aparece que ganó el jugador porque Piedra vence a Tijera.

Al terminar, se pregunta si se desea jugar de nuevo. Al elegir Sí, comienza otra partida desde las instrucciones. Al elegir No, el juego se cierra. También se puede cancelar la entrada para salir.

Organización del código

El archivo juego.py se encarga de las ventanas, la elección al azar y la repetición. El archivo logica.py contiene las tareas de revisar la opción, escribir el nombre de cada jugada y obtener el resultado. Estas tareas reciben las jugadas y devuelven una respuesta, sin abrir ventanas ni elegir al azar.

Para aplicar lo trabajado en la cuarta unidad, el programa usa la misma función para escribir el nombre de las dos jugadas. Las reglas siguen siendo las del diagrama del Autónomo 1. El PDF y el código del Autónomo 2 se toman como el avance anterior del proyecto.

Pruebas y resultados

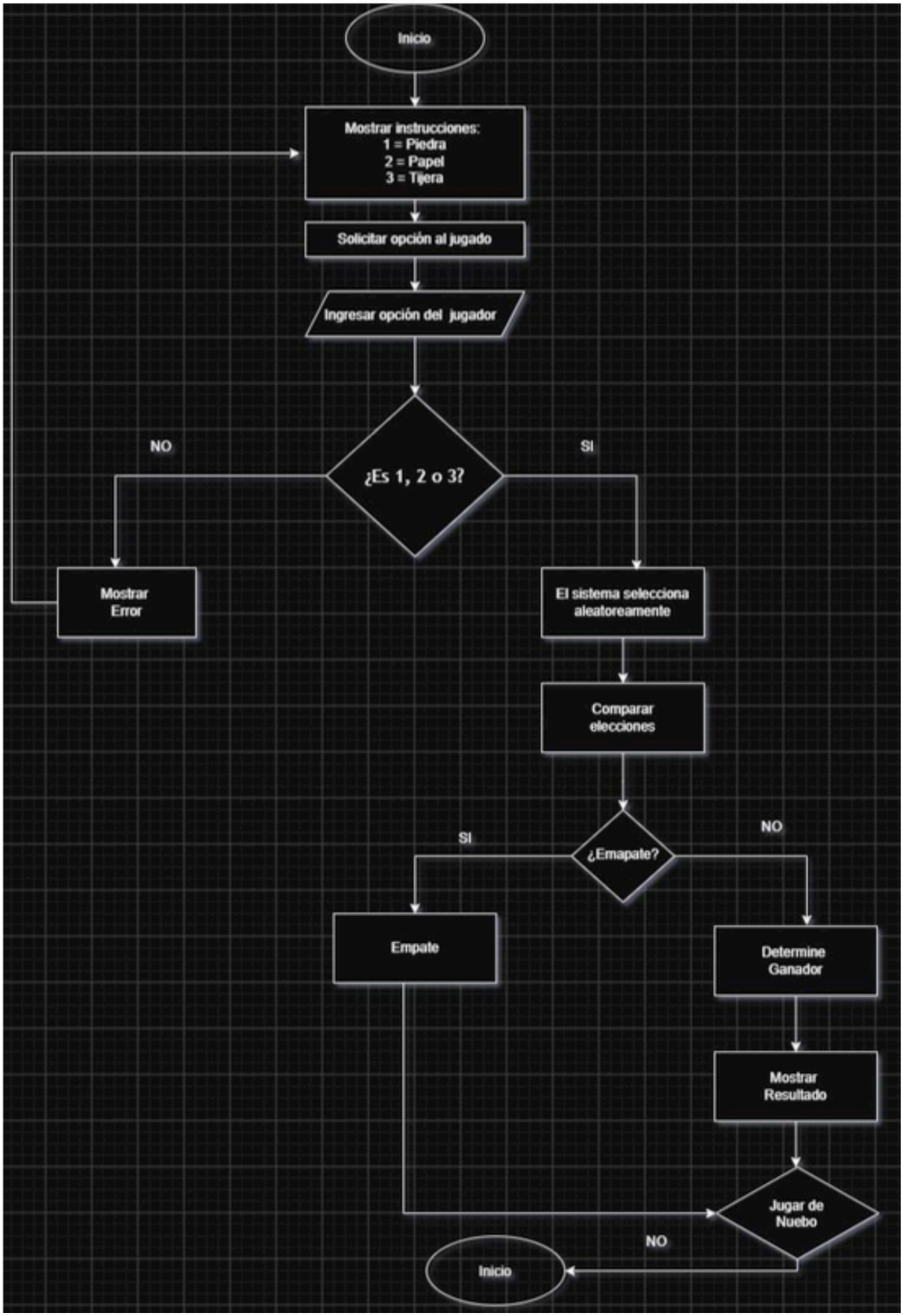
Durante la revisión se probaron las nueve parejas de jugadas con respuestas preparadas, para comprobar cada resultado sin esperar a que apareciera al azar. Los resultados fueron los siguientes:

Jugador	Computadora	Resultados Comprobados
Piedra	Piedra	Empate
Piedra	Papel	Gana la computadora
Piedra	Tijera	Gana el jugador
Papel	Piedra	Gana el jugador
Papel	Papel	Empate
Papel	Tijera	Gana la computadora
Tijera	Piedra	Gana la computadora
Tijera	Papel	Gana el jugador
Tijera	Tijera	Empate

Las nueve parejas dieron el resultado esperado: tres empates, tres victorias del jugador y tres victorias de la computadora. También se comparó el código del Autónomo 2 con la versión final y se mantuvieron los mismos ganadores.

Además, se revisaron respuestas incorrectas, cancelar, repetir y salir. Los datos incorrectos regresan a las instrucciones. La opción de repetir comienza otra ronda y la salida termina el programa.

Diagrama del Autónomo 1



Reflexión sobre el impacto de la tecnología

Este proyecto permite ver cómo la tecnología puede transformar una actividad conocida en un programa que responde a nuestras acciones. En este caso, el juego muestra de inmediato si hubo empate o quién ganó. La explicación del resultado ayuda a relacionar las reglas del juego con las decisiones del programa.

También permite practicar de manera individual y repetir las partidas. Su utilidad como apoyo para aprender es una posibilidad; no se ha trabajado con un grupo de estudiantes para comprobar si mejora su aprendizaje.

Para utilizarlo se necesita un equipo con Python y Tkinter. Esto puede ser una dificultad para alguien que no tenga una computadora o no sepa instalar el programa. Por otra parte, el juego no pide datos personales ni necesita Internet durante la partida.

Limitaciones y posibles mejoras

La versión actual es sencilla. No guarda las partidas ni tiene un marcador. Escribir números permite revisar los errores, aunque unos botones podrían hacer el juego más cómodo.

Tampoco se ha comprobado su uso con personas que necesiten ayuda para leer o manejar las ventanas.

Más adelante se podría agregar un marcador y botones para elegir cada jugada. Para saber si el juego sirve como apoyo al aprendizaje, también haría falta observar cómo lo usan otras personas y preguntarles qué comprendieron.

Conclusión

El proyecto reúne el diseño del Autónomo 1 y el programa del Autónomo 2. La versión final mantiene las reglas y organiza algunas tareas en funciones. Las pruebas muestran que las nueve parejas producen el resultado esperado. El trabajo permite entender cómo un diagrama puede guiar la creación de un programa sencillo.

Fuentes de apoyo

Celi Paladines, A. I. (2026). Aprendizaje Autónomo 1 y Aprendizaje Autónomo 2. Python Software Foundation. Documentación de Tkinter y selección aleatoria. GitHub. Inicio rápido para repositorios.

Link del video: